Reduction Clause

+

Trapezoidal rule in OpenMP

Reduction Clause

The **reduction clause** in **OpenMP** is used to safely combine values computed by multiple threads into a **single shared result**.

It eliminates **race conditions** when several threads update the same variable.

Calculate the sum of an array using multiple threads.

Initial sum = 10

Thread 1 reads 10
Thread 2 reads 10

Thread 1 adds 5 → 15
Thread 2 adds 8 → 18

Final sum = 18

Race Condition.

Ans: 10 + 5 + 8 = 23

Reduction Clause

Syntax: *#pragma omp parallel for reduction(operator : variable)*

Each thread gets its **own private copy** of sum.

Array = {2,4,6,8,10,12}

Total Sum = 42

Using **3 Threads**

Thread 0 Sum (2 + 4) = 6

Thread 1 Sum (6 + 8) = 14

Thread 2 Sum (10 + 12) = 22

OpenMP automatically performs Global Sum = 6 + 14 + 22 = 42

```
int sum = 0;

#pragma omp parallel
{
   int private_sum = 0;

   #pragma omp for
   for(int i=0;i<n;i++)
   {
      private_sum += A[i];
   }

   #pragma omp critical
   sum += private_sum;
}
```

reduction(+:sum)

Reduction Clause

Syntax: *#pragma omp parallel for reduction(operator : variable)*

```
#pragma omp parallel for reduction(+:sum)
#pragma omp parallel for reduction(*:product)
#pragma omp parallel for reduction(max:largest)
```

Reduction Clause

Common Reduction Operators

Operator	Meaning	Example
+	Addition	Sum of array
*	Multiplication	Product
-	Subtraction	Difference (rare)
&	Bitwise AND	Binary operations
|	Bitwise OR	Binary operations
^	Bitwise XOR	Binary operations
&&	Logical AND	Boolean operations
||	Logical OR	Boolean operations
max	Maximum value	Largest element
min	Minimum value	Smallest element

Reduction Clause

Sum of Array

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int A[10]={1,2,3,4,5,6,7,8,9,10};
    int sum=0;

    #pragma omp parallel for
reduction(+:sum)
    for(int i=0;i<10;i++)
    {
        sum += A[i];
    }

    printf("Sum = %d\n",sum);

    return 0;
}
```

Product of Numbers

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int A[5]={1,2,3,4,5};
    int product=1;

    #pragma omp parallel for
reduction(*:product)
    for(int i=0;i<5;i++)
    {
        product *= A[i];
    }

    printf("Product = %d\n",product);

    return 0;
}
```

Maximum Value

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>

int main()
{
    int A[8]={12,45,7,89,23,56,91,40};
    int maxVal = INT_MIN;

    #pragma omp parallel for
reduction(max:maxVal)
    for(int i=0;i<8;i++)
    {
        if(A[i] > maxVal)
            maxVal = A[i];
    }

    printf("Maximum = %d\n",maxVal);

    return 0;
}
```

Reduction Clause

Example with Four Threads

```
#include <stdio.h>
#include <omp.h>

int main()
{
   int A[8];
   int sum = 0;

   for(int i=0;i<8;i++)
      A[i]=i+1;

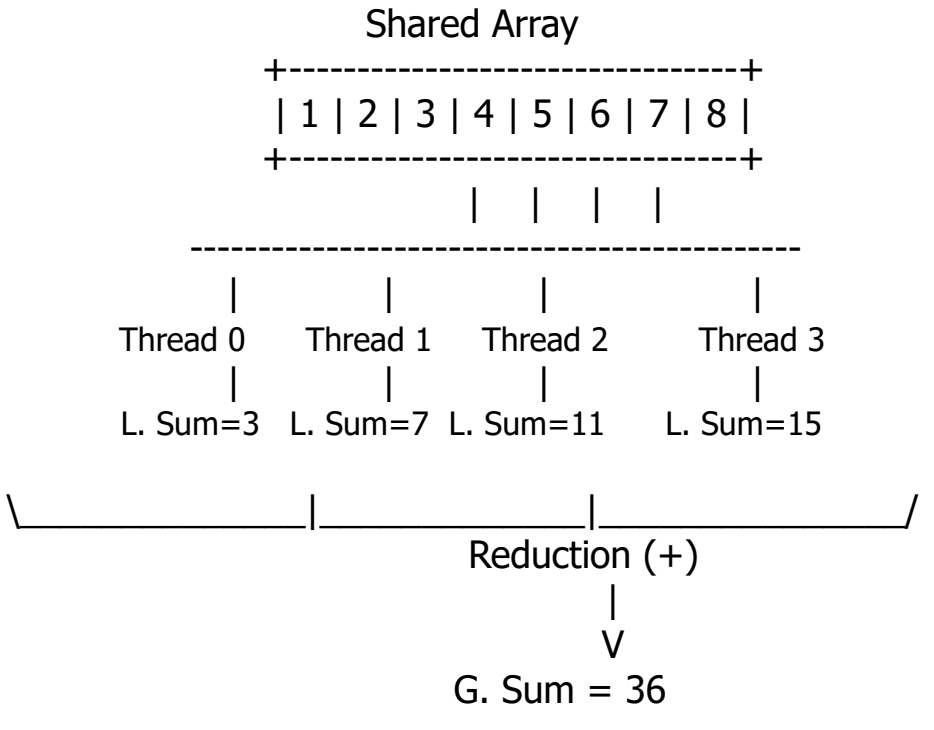
   omp_set_num_threads(4);

   #pragma omp parallel for reduction(+:sum)
   for(int i=0;i<8;i++)
   {
      printf("Thread %d processes A[%d]=%d\n",
            omp_get_thread_num(), i, A[i]);

      sum += A[i];
   }

   printf("\nFinal Sum = %d\n", sum);

   return 0;
}
```



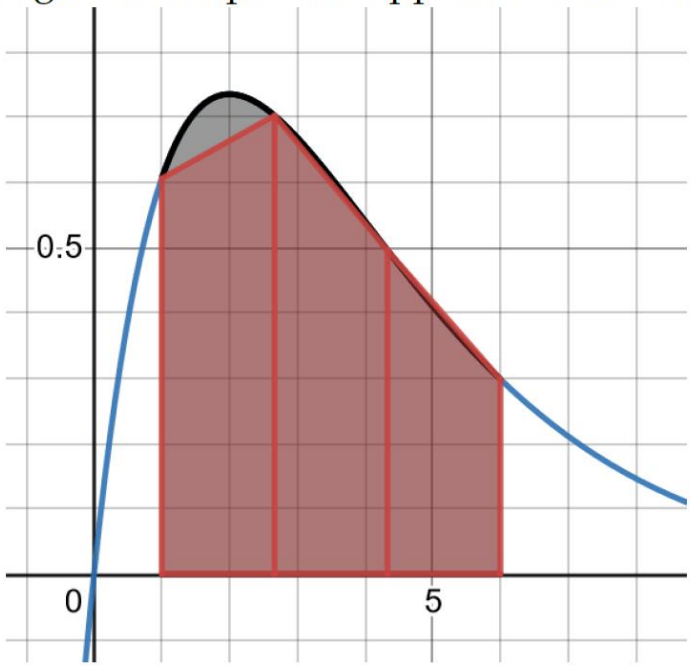
Trapezoidal rule in OpenMP

The **Trapezoidal Rule** is a numerical methods for approximating the value of a **definite integral.** It estimates the area under a curve by replacing the curved section with one or more **trapezoids.**

The trapezoidal rule works by estimating the area under a graph by a series of trapezoidal strips. In the figure below, we see an approximation to

$$\int_1^6 xe^{-0.5x}\,dx$$

using three strips. The approximated area is shown in red.



The accuracy increases as the number of strips increases.

Trapezoidal rule in OpenMP

For the definite integral

$$\int_a^b f(x)\,dx$$

the trapezoidal rule has the form

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[y_0 + 2y_1 + 2y_2 + \ldots + 2y_{n-1} + y_n\right] \qquad (1)$$

where

1. n is the number of strips and can be any number.

2. $y_n = f(x_n)$ are the values of $f(x_n)$ at the points x_i where $i = 0, 1, 2, \ldots, n$. Note that $x_0 = a$, $x_n = b$.

3. h is the width of each strip and

$$h = \frac{b-a}{n}.$$

4. $x_1 = a + h$, $x_2 = a + 2h$, $x_3 = a + 3h$,... and so on.

Trapezoidal rule in OpenMP

Approximate $\int_0^1 \sqrt{1+x^3}dx$ using the trapezoidal rule with 5 strips , to 3 decimal places.

In this question, $n = 5$, $a = 0$ and $b = 1$ so

h = (b - a) / n = 1/5 = 0.2

So using the formula above we get

$x_0 = 0$ and $y_0 = f(x_0) = 1.$

$x_1 = 0 + h = 0.2$ and $y_1 = f(x_1) = \sqrt{1+(0.2)^3} = 1.00399.$

$x_2 = 0 + 2h = 0.4$ and $y_2 = f(x_2) = \sqrt{1+(0.4)^3} = 1.03150.$

$x_3 = 0 + 3h = 0.6$ and $y_3 = f(x_3) = \sqrt{1+(0.6)^3} = 1.10272.$

$x_4 = 0 + 4h = 0.8$ and $y_4 = f(x_4) = \sqrt{1+(0.8)^3} = 1.22963.$

$x_5 = 0 + 5h = 1.0$ and $y_5 = f(x_5) = \sqrt{1+(1)^3} = 1.41421.$

Substituting into Substituting into (1) above we get:

Trapezoidal rule in OpenMP

$$\begin{aligned}\int_0^1 \sqrt{1+x^3}dx &\approx \frac{0.2}{2}\left[y_0+2y_1+2y_2+2y_3+2y_4+y_5\right]\\ &= 0.1\left[1+2\left(1.00399\right)+2\left(1.03150\right)\right.\\ &\qquad \left.+2\left(1.10272\right)+2\left(1.22963\right)+1.41421\right]\\ &= 1.11499.\end{aligned}$$

Hence, to three decimal places, $\int_0^1 \sqrt{1+x^3}dx \approx 1.115$.

Trapezoidal rule in OpenMP

Problem Statement

Approximate

$$\int_0^{10} x^2 \, dx$$

using

- Number of trapezoids = **1,000,000**
- Multiple OpenMP threads

Thread 0 : x1 ------ x250000

Thread 1 : x250001 - x500000

Thread 2 : x500001 - x750000

Thread 3 : x750001 - x999999

Each thread computes a **partial sum**.

OpenMP then combines them using a **reduction**.

Trapezoidal rule in OpenMP

Define the Function

```
double f(double x)
{
    return x * x;
}
```

Define the Interval

```
double a = 0.0;
double b = 10.0;
```

$$\int_0^{10} x^2\,dx$$

Compute Step Size

```
double h = (b - a) / n;
```

Final Formula

integral=(h/2.0)*(f(a)+2.0*sum+f(b));

Parallel Summation

```
#pragma omp parallel for reduction(+:sum)
for(int i=1;i<n;i++)
{
    double x=a+i*h;
    sum+=f(x);
}
```

Trapezoidal rule in OpenMP

```
#include <stdio.h>
#include <omp.h>

        double f(double x) // Function to integrate: f(x) = x^2
        {    return x * x;    }

int main()
{
   double a = 0.0;        double b = 10.0;     int n = 1000000;          // Number of trapezoids
   double h = (b - a) / n;    double sum = 0.0;       double integral;

   omp_set_num_threads(4);          // Set exactly 4 threads

   printf("Number of Threads = %d\n\n", omp_get_max_threads());

  #pragma omp parallel for reduction(+:sum) // Parallel computation of interior points
   for (int i = 1; i < n; i++)
   {
      int tid = omp_get_thread_num();   // Thread ID (0-3)
      printf("Thread %d computes i = %d\n", tid, i); // to see each thread executes each iteration

      double x = a + i * h;
      sum += f(x);
   }

   integral = (h / 2.0) * (f(a) + 2.0 * sum + f(b)); // Apply Trapezoidal Rule
   printf("Approximate Integral = %.12lf\n", integral);
   return 0;
}
```